The Architecture of Errors: From Universal Impossibility to Patch-Local LLM Reliability

Anonymous Author(s)
Affiliation
Address
email

Abstract

Universal LLM reliability is not a finite-library problem: across all possible tasks, tools, schemas, knowledge sources, and evaluator expectations, new intervention-distinguishable failure modes can appear without bound, so no finite intervention dictionary can guarantee bounded residual error universally. But deployed systems do not operate over the whole universe; they operate inside bounded patches (legal review, medical RAG, code repair, customer-support agents, contract extraction) with recurring tasks, schemas, tools, and evaluator expectations. Within such patches, empirical evidence suggests failures are sparse, repetitive, and concentrated in a small recurring catalogue, so reliability becomes a local catalogue-discovery and intervention-coverage problem rather than an exponential token-length problem. We make this transition formal with two propositions: a negative result that no universal finite intervention dictionary exists, and a positive result that under logarithmic or capped mode discovery the patch-local intervention budget grows polylogarithmically in sequence length and becomes domain-constant once the patch catalogue saturates. The framework relocates rather than dissolves long-context difficulty: where the number of hard decisions itself grows with task length, reliability remains hard; the contribution is to identify the on-axis intervention rather than to make those regimes easy.

1 Introduction

The standard worry about long-context autoregressive generation is exponential. If every token has independent error probability e, then after n tokens the chance of a fully correct output is $(1-e)^n$, which collapses to zero for any nontrivial e and large enough n [LeCun, 2023, Dziri et al., 2023]. Earlier work in this series [Anonymous, 2025] argued that this worry is misplaced because errors are not distributed uniformly across tokens: only 5–10% of tokens are "key" (genuinely dependent on long-range context [Fang et al., 2025]), while the remaining majority become nearly deterministic once enough context accumulates. Replacing $(1-e)^n$ with the two-rate model $P(\text{correct}) = (1-e_{\text{key}})^k(1-e_{\text{non}})^{n-k}$ with $k \ll n$ scaling sublinearly in n recovers the observed long-context coherence and converts the reliability question from "how does n grow?" to "how does k grow?".

From sparse tokens to recurring patterns. This paper takes the next step. Sparsity tells us *where* errors live; the natural follow-up is *what* they are. Recent failure-mode atlases supply a striking empirical answer: errors are not only sparse, they are also *repetitive*. ErrorAtlas [Ashury-Tahan et al., 2026], covering 83 models across 35 datasets, organises observed failures into 17 named categories sorted by prevalence, with a long-tailed distribution heavily concentrated in the head. In code, two error types (`AssertionError` and `NameError`) cover 86.35% of failures on HumanEval across 14 LLMs, replicated across 23 models on HumanEval Pro and MBPP Pro [Wen et al., 2024]. In math, MWPES-300K [Sun et al., 2025] categorises 304,865 errors from 15 LLMs across four math word-problem datasets and reports that dataset characteristics shape error patterns systematically. The same picture appears in multi-hop QA [Zhang et al., 2026], agentic tool use [Cemri et al., 2025], and retrieval-augmented generation [Wood and Forbes, 2024]. These observations suggest a third layer in the reliability architecture, complementing key-token sparsity (Layer 1) and within-key stratified-manifold structure (Layer 2 in Anonymous 2025). Within the small set of key tokens, only a fraction β produce *hard* failures, and inside bounded deployment patches those hard failures appear to cluster into a finite or effectively capped catalogue of recurring modes whose size grows much more slowly than the number of observed events.

Contributions. The central contribution is a shift in what counts as the reliability object. Universal LLM reliability is not a finite-library problem; patch-local reliability is a catalogue-discovery and intervention-coverage problem. Two propositions formalise the move. Proposition 1 is the negative result: in an unbounded domain that keeps producing intervention-distinguishable failures, no finite intervention dictionary can guarantee bounded residual error universally. Proposition 2 is the corresponding positive result inside a fixed deployment patch, $m \geq \lceil |C_{\text{eff}}|^{1-\varepsilon/e_{\text{hard}}} \rceil$, with a doubly-logarithmic sequence-length rate as the optimistic pre-cap special case and a domain-constant budget once the patch catalogue saturates. The sequence-level analogue is strictly tighter and approaches full-catalogue coverage as k grows; we surface that asymmetry rather than bury it.

Around this transition the paper does four supporting things. A three-layer framework, comprising sparsity (α), hard-token stratification (β), and patch-local mode catalogue ($|C_D|$), separates *where* errors occur, *what* recurring forms they take, and *which* capability interventions address them (§3). Logarithmic mode discovery is stated as an empirical postulate, not a theorem, calibrated against ErrorAtlas, HumanEval, and MWPES with $\sigma \in [0.87, 1.85]$ across anchors and $\sigma \approx 1.85$ carried as a conservative planning value. Section 4 synthesises evidence for failure clustering, cluster-selective interventions, and sublinear length scaling, drawing on $\sim$60 prior published results including the six-axis capability-elimination harvest of 28 quantitatively-anchored citations stratified into Patterns A/B/C (Appendix B). And the most-cited steep-decay counter-evidence (Appendix C) is re-audited and shown to decay primarily over task-structure variables, including compositional graph size, fact count, log-time horizon, capacity threshold, and evidence scope, rather than raw token length.

2 Related Work

Failure-mode taxonomies. ErrorAtlas [Ashury-Tahan et al., 2026], MWPES-300K [Sun et al., 2025], the HumanEval categorisation of Wen et al. [2024], and the RFMDataset [Guo et al., 2025] together suggest that, at any given corpus size, the number of distinct named failure modes is small (typically 8–20) and the cumulative coverage of the top modes is high. Domain-specific taxonomies for multi-hop QA [Zhang et al., 2026], multi-agent systems [Cemri et al., 2025], and tool-augmented agents [Yao et al., 2024] report the same pattern.

Targeted interventions. Each named cluster has been the subject of focused intervention research. Python execution closes most of the arithmetic-error cluster [Gao et al., 2023a, Chen et al., 2023, Gou et al., 2024b]; constrained decoding eliminates format violations [Suresh et al., 2025, Wang et al., 2025b, Dong et al., 2025]; execution feedback closes most code-logic errors [Shinn et al., 2023, Huang et al., 2023]; process reward models reduce step-level reasoning errors [Wang et al., 2024b, Lightman et al., 2024]; RAG strongly reduces hallucinations [Wood and Forbes, 2024]; preference optimisation closes over-refusal [Karaman et al., 2024]; structured uncertainty fixes most tool-call failures [Suri et al., 2025]. Two structural observations recur: each intervention is *cluster-selective* (residuals belong to a different named cluster, not to the targeted one), and *additivity is approximate but not perfect* [Patel et al., 2026, Le, 2026].

Length-decay benchmarks. A parallel literature measures the *shape* of the reliability decay curve. Mild-decay results (Loong [Wang et al., 2024a], GSM-∞ [Zhou et al., 2025], RULER [Hsieh et al., 2024], anchor-based LLMs [Pang et al., 2024]) report log-linear, sigmoidal, or threshold-like decay inconsistent with smooth $(1-\varepsilon)^n$. Steep-decay results [Dziri et al., 2023, Kuratov et al., 2024, Kwa et al., 2025, Wan et al., 2026] are sometimes cited as exponential-compounding evidence. The apparent tension is reduced on careful reading: every steep-decay paper decays over a variable distinct from raw token length, which Appendix C documents.

Gap. What is missing is a quantitative bridge between the small, repeating taxonomy catalogue and a polylog intervention budget. We provide that bridge in §3.

3 Theoretical Framework

Four levels of $|C|$. Before any proposition, a definitional setup. LLM error analysis routinely conflates four objects that the rest of this paper needs to keep apart:

- **L1: failure events.** Concrete LLM errors observed in a benchmark; the raw data behind any taxonomy.
- **L2: empirical taxonomy categories.** Researcher-chosen labels grouping L1 events. ErrorAtlas's 17 categories, MWPES's top-4 classes, and HumanEval's `AssertionError`/`NameError` partition all live here. L2 is observable but depends on the taxonomer's resolution choices.
- **L3: latent failure modes.** The unobserved underlying clusters that L2 approximates. L3 is the quantity Postulate 1 is morally about, but it cannot be measured directly; we treat L2 as a noisy proxy for L3.
- **L4: capability axes / interventions.** The engineering unit: a Python interpreter, a constrained decoder, a retrieval-augmented generator. L4 is coarser than L2 in practice; one capability axis typically targets several L2 categories at once (§4, Claim B).

Throughout this section, $|C|$ refers to the **L2** count: that is what published taxonomies report. Postulate 1 is therefore *engineering*-relevant because L4 is coarser than L2 (so a small intervention library can sweep across many categories at once), and *epistemically* conditional because L2 is a noisy proxy for the actual L3 mode catalogue whose faithfulness has not been independently measured. Tagging the formal claims at the right level keeps the framework honest where casual error-talk slips.

Roadmap. The framework separates three questions that are often conflated. First, *where* do errors occur? Only a sparse subset of key decisions. Second, *what* kinds of errors recur? A local catalogue of failure modes. Third, *what fixes them*? A smaller library of capability interventions. The two propositions of §3.4 formalise the transition from universal impossibility to patch-local tractability: there is no finite intervention dictionary that covers all possible deployments, but inside a fixed deployment patch the required library is small and slowly growing.

3.1 β-stratification of key tokens

The two-rate model of Anonymous [2025] distinguishes k key tokens (error rate e_{key}) from $n-k$ non-key tokens ($e_{\text{non}} \ll e_{\text{key}}$). Empirical atlases suggest that even within the key-token class errors are not uniformly distributed: most key tokens are "decisions" for which the model has stable representations; only a fraction concentrate the actual failures.

Definition 1 (Hard fraction)**.** Partition the k key tokens of a sequence into easy and hard subsets,

$$k_{\text{hard}} = \beta k, \qquad k_{\text{easy}} = (1-\beta)k, \qquad \beta \in (0,1),$$

where hard key tokens have an elevated error rate e_{hard} corresponding to manifold-transition decisions in the sense of Anonymous [2025, §3.2], and easy key tokens have $e_{\text{easy}} \approx e_{\text{non}}$.

Under Definition 1, the composed sequence-level reliability becomes

$$P(\text{correct}) \;=\; (1-e_{\text{hard}})^{\beta k}\,(1-e_{\text{easy}})^{(1-\beta)k}\,(1-e_{\text{non}})^{n-k}. \tag{1}$$

We do not assume iid token errors: the three rates summarise *conditional per-decision failure hazards* after conditioning on prior decisions being correct, and grouping by stratum yields the multiplicative survival expression. Because $e_{\text{easy}} \approx e_{\text{non}}$ once context accumulates, the failure rate is dominated by the βk hard tokens, and we treat e_{hard} as the load-bearing quantity. The parameter β is *latent*: failure-mode atlases report $\Pr(\text{category} \mid \text{error occurred})$, which does not identify $\Pr(\text{hard} \mid \text{key-token decision})$. We carry β symbolically.

3.2 Empirical postulate: logarithmic mode discovery

Stratification names a target, the βk hard-token decisions, but does not yet tell us how many distinct ways those hard decisions can fail. The empirical answer from Section 1 is that failures repeat: a small number of recurring patterns covers most of the mass. The next question is how the catalogue's size grows with the number of observed failures, because that is the quantity an intervention library has to keep up with.

Two candidates exist in the type–token literature: Heaps' law (power-law, $|C| \approx K \cdot k_{\text{hard}}^{b}$ with $b \in [0.4, 0.6]$ [Manning et al., 2008]), and logarithmic discovery. **Zipfian rank-frequency does not imply logarithmic growth.** Heaps' law is the correct type–token consequence of Zipf-distributed events, and it is power-law, not logarithmic. We therefore state logarithmic mode discovery as an empirical postulate, defensible by direct measurement of error taxonomies, not as a theorem.

Two sample-size variables matter, and they are easy to conflate. Let $h(n) = \beta k(n)$ denote the number of hard-token decisions in a single sequence of length n; let T denote the number of observed hard-failure events in a corpus used to discover the domain catalogue. These are not the same object: $h(n)$ controls per-sequence exposure, T controls empirical discovery. We write C_D for the full reachable catalogue inside domain patch D, $C_{\text{seen},D}(T)$ for the subset discovered after T sampled failures, and $C_{\text{active},D}(n)$ for the subset a single sequence of length n can activate.

Postulate 1 (Patch-indexed catalogue discovery)**.** Within a fixed application domain D, the number of named recurring failure modes discovered after T sampled hard-failure events is bounded by

$$|C_{\text{seen},D}(T)| \;\le\; \min\bigl(A_D + \sigma_D \ln T,\ |C_D|\bigr), \qquad \sigma_D > 0,\ A_D \ge 0.$$

The cap $|C_D|$ is the domain-imposed ceiling. The postulate concerns catalogue *discovery*, not the number of hard decisions inside a single sequence.

Assumption 2 (Per-sequence active-mode exposure)**.** For a single sequence of length n in domain D,

$$|C_{\text{active},D}(n)| \;\le\; \min\bigl(A'_D + \sigma'_D \ln h(n),\ |C_D|\bigr).$$

Primed constants are distinct from those of Postulate 1: corpus discovery and per-sequence activation need not share the same rate.

The two bounds answer different questions, and the rest of the paper picks the right one at each point. Proposition 2's sequence-length scaling rate is governed by $C_{\text{active},D}(n)$ via Assumption 2: how much of the catalogue can a single sequence touch? Full domain-library budgeting, the practical engineering target for a deployed system, uses C_D: how large does the library need to be to cover the domain's reachable failure modes? The cap $|C_D|$ enters both: once enough has been sampled (or activated) to saturate the domain ceiling, further growth stops and the budget becomes a domain-constant function of $|C_D|$ alone.

Empirical calibration. Existing LLM error taxonomies provide endpoint anchors for small named catalogues at observed corpus scales. ErrorAtlas, HumanEval-style code taxonomies [Wen et al., 2024], and MWPES [Sun et al., 2025] place $|C|$ roughly in the 8–20 range across corpora ranging from approximately 10^4 to 3×10^5 observed failures, yielding $\sigma \in [0.87, 1.85]$ under the simple $A = 0$ logarithmic calibration. We use $\sigma \approx 1.85$ as a conservative planning value, not as a fitted law. These endpoint counts are not discovery curves: the missing test is a subsample-vs-discovered-modes measurement within a fixed deployment patch. Full calibration details are in Appendix E.

Capabilities are coarser than error categories. The category count $|C|$ is conservative for engineering because deployed interventions operate at the capability level, not the label level. A Python interpreter can remove execution-error components across arithmetic, unit conversion, counting, list manipulation, and date arithmetic; a constrained decoder can eliminate several structural output failures at once. The formal coverage problem is therefore closer to weighted set cover over failure mass than one-intervention-per-category counting. The ranked-category model used in Proposition 2 is a tractable proxy; the full capability harvest appears in Appendix B.

Domain patches cap the engineering problem. A fixed deployment patch restricts the task family, schemas, tools, workflows, and evaluator expectations. Prior work on localized representations [Park et al., 2024, Li and Sarwate, 2025], cross-domain performance variation [Wang et al., 2024c], and long-tail knowledge [Mallen et al., 2023, Kandpal et al., 2023] supports the weaker claim that model behaviour is strongly domain-dependent; it does not directly measure $|C_D|$. We therefore treat the patch's reachable catalogue as finite or effectively capped as a modelling assumption motivated by domain heterogeneity, not as a theorem derived from boundedness. The full evidence breakdown is in Appendix D.

3.3 Coverage by a targeted intervention library

With the catalogue defined, the next question is how much residual error a library of m targeted interventions actually closes off. Given a catalogue of $|C|$ recurring modes, let $p_1 \geq p_2 \geq \cdots \geq p_{|C|}$ denote the ranked empirical hard-error mass of those modes with $\sum_i p_i = 1$, so a library covering the top m modes removes cumulative error mass $\sum_{i=1}^{m} p_i$. We adopt the log-coverage form

$$F(m; |C|) \;=\; \min\left(1,\; \frac{\ln m}{\ln |C|}\right) \tag{2}$$

as a *postulated* parametric approximation to that cumulative top-m mass, not as a raw count of categories covered. Reading the formula: the first few interventions cover disproportionate mass (the head of the distribution is dense), then incremental coverage falls off as the library saturates the catalogue. For our ErrorAtlas anchor ($|C| = 17, m = 5$), $F(5; 17) \approx 56.8\%$; doubling the library to $m = 10$ pushes it to $F(10; 17) \approx 81.3\%$.

This is an empirical best-fit choice, not derived from Zipf. ErrorAtlas sorts its 17 categories by prevalence but does not publish a single cross-domain cumulative top-m curve at this writing; alternative cumulatives (Zipf–Mandelbrot, truncated power law, saturating exponential) match the same qualitative shape with comparable freedom. Appendix A.5 shows the polylog conclusion of §3.4 survives across the family.

Two caveats matter for what comes next. (a) F tracks cumulative error mass across ranked L2 categories, not L4 capability axes deployed; in real deployments, where one capability often sweeps several L2 categories, residual error at a given m is generally lower than (2) predicts. (b) The form is intended for $m \geq 2$, $|C| \geq 2$; we adopt $F(0; |C|) := 0$ and $F(1; |C|) := p_1$ (the mass of the single most prevalent mode) since the log expression is not anchored at $m = 1$. After deploying the top-m library, the residual per-hard-token error rate is

$$e_{\text{res}}(m) \;=\; \big(1 - F(m; |C|)\big)\, e_{\text{hard}}. \tag{3}$$

3.4 From universal impossibility to patch-local reliability

The finite-catalogue claim is patch-local. It does not say that all possible LLM failures can be covered by a universal list of fixes. Across all possible tasks, tools, schemas, knowledge sources, workflows, and evaluator expectations, new intervention-distinguishable failures can keep appearing. In that setting, a finite intervention dictionary is not a well-posed reliability target.

The positive result begins only after a deployment patch D has been fixed. Inside a patch, the task family, schemas, tools, evaluator expectations, and admissible workflows recur. The engineering question then changes from "Can one library cover all possible LLM failures?" to "How large must the local library be to cover enough of the reachable failure catalogue C_D?"

Proposition 1 (No Universal Finite Intervention Dictionary)**.** *Fix a residual-error tolerance ε. If a domain D contains an infinite sequence of failures where each new failure is not covered by any finite intervention dictionary covering all earlier failures, then the ε-resolution failure catalogue C_D^{ε} is infinite. Consequently, no finite intervention dictionary can guarantee residual error below ε over all of D.*

Proof sketch. Each new failure is intervention-distinguishable from the failures before it. The sequence therefore generates infinitely many distinct intervention modes, and any finite dictionary must miss some later mode. The full proof, including the precise definition of mode-level coverage, is in Appendix A.1.

Engineering meaning. Universal reliability is not a finite-library problem. A fixed list of interventions cannot cover open-ended LLM use in general. The rest of the paper is therefore about *patch-local* reliability, not universal reliability. Proposition 1 is the inoculation against a misread: we are not claiming a fixed list of ≈ 50 named patterns covers LLM use in general.

The positive result composes β-stratification (Definition 1), the active-mode bound (Assumption 2), and the log-coverage form (Eq. 2) into a local budget statement. The result is conditional engineering math, not an unconditional theorem about LLMs: it depends on the log-coverage approximation, the active-mode exposure assumption, and the patch hypothesis that C_D is finite or effectively capped. Appendix A.2 states these conditions explicitly before the derivation.

Proposition 2 (Patch-Local Intervention Budget)**.** *Fix a deployment patch D. Let $\varepsilon \in (0, e_{hard})$ be the target per-hard-token residual error rate. Under the log-coverage approximation of §3.3, a library covering the dominant local modes satisfies the target once*

$$m \;\geq\; \left\lceil |C_{\mathit{eff}}|^{1-\varepsilon/e_{hard}} \right\rceil, \tag{4}$$

where C_{eff} is either the active catalogue $C_{\mathrm{active},D}(n)$ touched by a single sequence, or the full reachable catalogue C_D of the deployment patch. If $C_{\mathrm{active},D}(n)$ grows logarithmically with the number of hard decisions and $k(n) = \Theta(\log n)$, then the pre-cap intervention budget grows doubly-logarithmically in sequence length. Once the patch catalogue saturates, the required budget becomes domain-constant in $|C_D|$.

Proof sketch. The library removes cumulative hard-error mass $F(m; |C|)$. The residual hard-token error rate is therefore $(1-F)\, e_{\text{hard}}$. Requiring this residual to be at most ε rearranges, after substituting the log-coverage form, to the stated bound. The full step-by-step derivation, including the pre-cap and cap regimes, is in Appendix A.2.

Engineering meaning. Once the patch is fixed, the problem changes. The relevant engineering question is no longer whether arbitrary future failures exist, but how quickly the reachable local catalogue is discovered and how much hard-error mass is removed by the top interventions. The intervention budget for per-hard-token reliability is small and slowly growing, domain-constant in the cap regime, with the exact size determined by the local discovery and rank-coverage curves rather than by a universal prior.

Sequence-level caveat. Proposition 2 bounds residual error per hard decision. A one-shot sequence-level target is strictly stricter because many hard decisions occur in one output: as k grows the allowable residual per hard decision shrinks, and the required library approaches full-catalogue coverage. In some regimes the non-hard-token error mass alone already exceeds the sequence-level budget, so hard-token interventions cannot meet the SLA by themselves. The full three-regime analysis (and the per-hard-token tolerance τ_{seq} that converts a sequence target into the Proposition 2 bound) is in Appendix A.3.

A common engineering rule of thumb, "fix ≈ 50 patterns per domain $\rightarrow \approx 90\%$ error reduction," is best understood as a per-hard-token planning prior. The analogous sequence-level claim requires correspondingly more interventions and tighter coverage of the tail.

We call these statements propositions rather than theorems because their force is conditional: they formalise the consequences of the paper's modelling assumptions rather than deriving a universal law of LLM reliability. The doubly-logarithmic rate is the optimistic special case under logarithmic active-mode exposure; Appendix A.5 gives the Heaps and saturation variants. Appendix A.4 expands on the conditional reading.

4 Empirical Evidence

We collect evidence for three load-bearing claims: errors cluster into a small recurring set (A); each cluster is addressable by one targeted intervention (B); reliability decays sublinearly with output length (C).

Claim A: Failure-mode clustering. The strongest single result is ErrorAtlas [Ashury-Tahan et al., 2026]: 83 models $\times$ 35 datasets, $\approx 7{,}000$ failures, 17 named categories with a long-tailed, head-concentrated prevalence ordering. Domain-specific Paretos reproduce the shape inside each domain: math (MWPES top-4 categories dominate per model with strong cross-model overlap [Sun et al., 2025]), code (`AssertionError+NameError` = 86.35% on HumanEval, replicated across 23 models [Wen et al., 2024]), multi-hop QA (a single dominant "missing-evidence" mode [Zhang et al., 2026]), agentic tool use (under 20 recurring modes across two studies [Cemri et al., 2025, Yao et al., 2024]), and RAG (a handful of distinguishable hallucination types [Wood and Forbes, 2024]). The catalogue is also *stable* across models: ErrorAtlas reports a fixed category hierarchy across 83 models; RFMDataset [Guo et al., 2025] finds "strikingly similar failure mode distributions" across ten advanced reasoning models; EDIT [Dai et al., 2025] measures a $\approx 4.7\%$ key-step fraction stable across models. Schaeffer et al. [2025] provide the closest theoretical anchor, proving that the observed aggregate power-law scaling of LLM evaluations requires the per-problem success-rate distribution to be heavy-tailed near $p = 0$, structurally related to Postulate 1.

Claim B: Cluster-selective capability interventions. A dedicated harvest yields 28 capability-elimination citations across six independent axes (arithmetic, code execution, format/structure, perception/grounding, knowledge/RAG, verification), each axis independently confirmed by between three and nine citations. The full table and the three structural patterns (Pattern A by-construction, B strong-empirical-with-class-shift, C moderate-with-shift) appear in Appendix B.

Arithmetic is the cleanest case: PAL [Gao et al., 2023a] lifts GSM-Hard from 20.1% to 61.5%, and the residuals are problem-comprehension errors rather than arithmetic ones. Format violations admit something stronger still: constrained decoding zeroes invalid-token probability by construction [Suresh et al., 2025, Wang et al., 2025b, Dong et al., 2025]. Code-logic errors yield to execution feedback (Reflexion + AgentCoder push HumanEval pass@1 from 80% to 96.3%, with residuals in spec-misinterpretation [Shinn et al., 2023, Huang et al., 2023]); reasoning steps to process supervision (Math-Shepherd: Mistral-7B MATH 28.6% $\rightarrow$ 43.5% [Wang et al., 2024b]); hallucinations to RAG, where Acurai [Wood and Forbes, 2024] reports 100% elimination on RAGTruth, a benchmark-conditional empirical result rather than a general by-construction guarantee. Two further clusters complete the picture: POROver lifts the not-overrefused rate from 57.6% to 82.1% [Karaman et al., 2024], SAGE-Agent raises When2Call accuracy from 36.5% to 65.2% [Suri et al., 2025].

The class-shift signature, in which post-intervention residuals belong to structurally different classes, is the empirical content of the cluster-orthogonality claim that underwrites Proposition 2's composition step. Capabilities are coarser than error classes: a single Python interpreter removes execution-error components from five named clusters; constrained decoding eliminates format and the structural component of "missing required element" jointly. A negative control sharpens the selectivity claim. DebugBench [Tian et al., 2024] reports that execution feedback works for syntax and reference errors and is explicitly "unhelpful for logic errors," exactly where the framework predicts capability provisioning fails.

Claim C: Sublinear length scaling. Direct evidence for sublinear (rather than exponential) decay shows up across very different measurement designs. Loong [Wang et al., 2024a] reports GPT-4o on Chain-of-Reasoning declining $81.6\% \rightarrow 32.9\%$ across the $10\text{K} \rightarrow {>}200\text{K}$ context bins; log-linear over $\log L$ and not well explained by smooth independent per-token exponential decay. GSM-∞ [Zhou et al., 2025] attacks the independent-error model from a different angle: exponentially more inference compute yields only *linear* AUC gains, and DeepSeek-R1 maintains $10\%+$ accuracy at 130 reasoning operations where $(1-\varepsilon)^{130}$ for any reasonable ε predicts near-zero. Press et al. [2023] find a $\approx 40\%$ compositionality gap that is approximately *constant* across the GPT-3 family, direct evidence against per-step independent compounding (which would predict the gap to grow with chain length).

Structural correlates of the same phenomenon appear elsewhere. Pang et al. [2024]'s Anchor LLMs achieve $\approx 99\%$ K/V reduction with only minor accuracy compromise: the effective number of distinct attention-key vectors a model needs is far smaller than n. Think-Prune-Train [Costello et al., 2025] raises Pass@1 while Pass@20 plateaus, the manifold-transition signature. RULER [Hsieh et al., 2024] finds threshold behaviour inconsistent with smooth $(1-\varepsilon)^n$. METR [Kwa et al., 2025] fits logistic-in-log-length, which is mathematically sublinear in length itself.

Self-consistency [Wang et al., 2023] is consistent with but does not *prove* clustered structure: Condorcet aggregation of iid Bernoulli chains can yield similar gain magnitudes, so we treat it as suggestive rather than diagnostic. Prominent steep-decay counter-evidence [Dziri et al., 2023, Kuratov et al., 2024, Wan et al., 2026, Karpinska et al., 2024] decays over variables distinct from raw token length; per-paper re-audits showing the relocation pattern are in Appendix C.

5 Practical Implications

Reliability engineering is local patch coverage. Within a fixed domain patch, Proposition 2 makes reliability a small-catalogue engineering problem rather than an asymptotic scaling problem. A team should initially budget for a library on the order of tens of interventions, then refine from the local mode-discovery curve $C_{\text{seen},D}(T)$ and the empirical rank-frequency distribution. As a rule of thumb consistent with the head-mass figures in ErrorAtlas, HumanEval, and MWPES, ≈ 50 named interventions cover the bulk of the per-hard-token failure distribution in many measured domains. This is a planning prior calibrated to current taxonomies, not a universal constant; the local mode-discovery curve sets the actual library size for any given deployment. The same base model in cardiology RAG, legal contract drafting, and code review yields three different libraries because C_D, A_D, σ_D all change with the deployment domain.

Per-hard-token vs. sequence-level targets matter for SLA design. Per-hard-token residual error, the right metric for continuous-correction systems, has a polylog budget. Sequence-level failure probability, the right metric for one-shot systems, is strictly stricter and approaches full-catalogue coverage as k grows. Production deployments should design SLAs to match the actual cost structure, not read the per-token result as the production SLA.

Library design as engineering, not asymptotic theory. An intervention library can be assembled module-by-module, in any order, as long as modules target distinct failure classes. Layer-separated interventions (constrained decoding + retrieval + process supervision + tool call) compose approximately additively in our data; the Le [2026] caveat applies only to interventions sharing a prompt channel. Library construction is therefore highly parallelisable.

Capability libraries are coarser than error-class libraries. Five named math classes (arithmetic, units, counting, list manipulation, date arithmetic) collapse under one Python interpreter; three code classes (SyntaxError, NameError, most TypeError) collapse under one execution-feedback loop; format violations and the structural component of "missing required element" (together more than 20% of ErrorAtlas) collapse under one constrained-decoder deployment. A team budgeting for ≈ 50 interventions per domain can expect to need fewer than 50 capability-axis interventions while covering ≈ 50 named clusters. The six axes of Appendix B are closer to the right unit of engineering accounting than the twelve clusters.

6 Discussion

By-construction elimination as a boundary case. A subset of the capability-elimination harvest deserves separate accounting. Seven of the 28 citations (Appendix B; constrained decoders [Suresh et al., 2025, Dong et al., 2025, Li et al., 2026, Zhang et al., 2023, OpenAI, 2024], proof kernels [Ren et al., 2025], static syntax checks) achieve residual error rate equal to zero *by mathematical construction*, not statistically. Constrained decoders set $P(\text{invalid token}) = 0$ at every generation step; the class of grammar-violating outputs is mathematically empty. In this regime the polylog bound of Proposition 2 is loose: covered clusters contribute exactly zero to residual error, not a small positive quantity. The result strengthens, rather than refutes, the framework. And Pattern A applies precisely to structural/verifiable classes (format, syntax, schema, invalid proofs), which sit at the head of the heavy-tailed cluster frequency distribution: the strongest mechanism lands exactly where the catalogue is densest.

Counter-evidence relocates, not dissolves. Five prominent steep-decay papers [Dziri et al., 2023, Kuratov et al., 2024, Kwa et al., 2025, Wan et al., 2026, Karpinska et al., 2024] are routinely cited as exponential-compounding evidence. Every one decays over a variable distinct from raw token length: compositional graph size [Dziri et al., 2023], number of supporting facts [Kuratov et al., 2024], log-time horizon [Kwa et al., 2025], capacity threshold [Wan et al., 2026], evidence scope [Karpinska et al., 2024]. The apparent rapid decay is, in each case, in a quantity our framework already concentrates the action in (k_{hard} and $|C|$), not in raw sequence length: a relocation, not a dissolution. These regimes remain hard; the framework's value is directing intervention toward capability provisioning along the actual decay axis rather than toward context-window expansion that does not help. Appendix C walks through each paper.

Limitations. Postulate 1 is empirical, not derived; a domain with genuinely Heaps-power-law mode discovery would invalidate the doubly-logarithmic special case while preserving the qualitative polylog conclusion. The coverage form of §3.3 is an empirical best-fit; readers should not interpret the numerical match against anchors as theoretical confirmation. Cluster-orthogonality is approximate; Le [2026]'s caveat on prompt-channel interference tightens the bound in shared-channel settings. Domain narrowness: Postulate 1's empirical anchor rests on three taxonomies (ErrorAtlas, HumanEval, MWPES), all published 2025–2026 and covering general/code/math; the framework is untested on agentic workflows, long-running scientific reasoning, and multi-turn tool use over millions of tokens, precisely the regimes where $|C|$ may grow faster than logarithmically. The $\sigma \approx 1.85$ estimate is a single-point calibration, not a discovery-curve fit; a subsample-vs-distinct-modes plot has not been published for any LLM error taxonomy at this writing. β is latent and we report no empirical range. Taxonomy granularity (the L2–L3 gap) is unmeasured. Patch-shift between deployments changes $A_D, \sigma_D, \beta_D, |C_D|$ simultaneously; libraries calibrated against one patch under-cover the next. Finally, the framework *relocates* the difficulty of long-context reliability rather than resolving it: where k_{hard} grows with task length (adversarial compositional structure, multi-hop chains, long agent horizons), reliability remains hard. The contribution is to name the on-axis intervention, not to make those regimes easy. A falsifiability test the framework passes: DebugBench [Tian et al., 2024] explicitly catalogues which classes execution-feedback can and cannot repair, and the empirical split (works for syntax/reference; "unhelpful for logic errors") is exactly what the framework's selectivity claim predicts.

7 Conclusion

LLM reliability is often framed as an asymptotic scaling problem. Universal reliability is not a finite-library problem (Proposition 1): an unbounded domain that keeps producing intervention-distinguishable failures cannot be covered by any finite dictionary. The contribution of this paper is to show that, once a bounded deployment patch is fixed, reliability becomes a local engineering problem. Prior work in this line [Anonymous, 2025] argued that the standard asymptotic framing rests on a false uniformity assumption: errors concentrate at ≈ 5–10% of tokens. This paper takes the next step: within that sparse set, errors are not only concentrated but also *repetitive*, clustering into a finite catalogue whose size grows logarithmically with observed failures under the empirical postulate of §3.2, or as a small power under the conservative Heaps alternative (Appendix A.5). The conditional consequence (Proposition 2) is that the intervention budget required to bound per-hard-token residual error scales polylogarithmically in sequence length within a fixed domain patch, and becomes a domain-constant once the patch ceiling $|C_D|$ is reached. Sequence-level reliability targets are strictly tighter. Available evidence is consistent with libraries on the order of tens of interventions covering the head of the per-hard-token failure distribution in many fixed domains. The exact number is patch-indexed and should be estimated from local discovery and rank-coverage curves, not assumed from cross-domain priors. This reframes the question from "can we bound the growth of n-token error?" to "have we catalogued enough failure modes *inside the deployment patch*?" The latter is finite and addressable once a bounded deployment patch has been fixed.

The open empirical question this paper invites is direct measurement of the mode-rate constant σ on new domains (agentic, scientific, code-in-production), and a corresponding sequence-level validation that measures S_{base}, β, and active-mode exposure directly in deployed systems. The conceptual follow-on goes in a different direction. This paper names the engineering object: a patch-local failure catalogue and the intervention budget that covers its head. It does not say how production systems actually accumulate and govern that library over time. That question, how the deployment-time scaffold (instructions, tools, retrieval, memory, orchestration, governance) learns to cover the local failure topology, is the subject of follow-on work in this line. Reliability engineering, in that frame, is governance of the scaffold rather than scaling of the weights.

References

Anonymous. Beyond exponential decay: Rethinking error accumulation in large language models. *arXiv preprint*, 2025. Author field anonymized for double-blind submission; arXiv ID public.

Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations (ICLR)*, 2024.

Shir Ashury-Tahan, Yifan Mai, Elron Bandel, Michal Shmueli-Scheuer, and Leshem Choshen. ErrorMap and ErrorAtlas: Charting the failure landscape of large language models. *arXiv preprint*, 2026.

Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why do multi-agent LLM systems fail? *arXiv preprint*, 2025. Introduces the MAST taxonomy (Multi-Agent System Failure Taxonomy).

Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *Transactions on Machine Learning Research (TMLR)*, 2023.

Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Yantao Li, Jianbing Zhang, and Zhiyong Wu. SeeClick: Harnessing GUI grounding for advanced visual GUI agents. In *Proceedings of ACL*, 2024.

Caia Costello, Simon Guo, Anna Goldie, and Azalia Mirhoseini. Think, prune, train, improve: Scaling reasoning without scaling models. *arXiv preprint*, 2025.

Chengwei Dai et al. Capture the key in reasoning to enhance CoT distillation generalization. In *Proceedings of ACL*, 2025. Earlier arXiv version titled "Beyond Imitation: Learning Key Reasoning Steps from Dual Chain-of-Thoughts in Reasoning Distillation".

Alex Dantart. Reliability by design: Quantifying and eliminating fabrication risk in LLMs. from generative to consultative AI: A comparative analysis in the legal domain and lessons for high-stakes knowledge bases. *arXiv preprint*, 2026.

Yixin Dong, Charlie F. Ruan, Yaxing Cai, Ruihang Lai, Ziyi Xu, Yilong Zhao, and Tianqi Chen. XGrammar: Flexible and efficient structured generation engine for large language models. In *Proceedings of MLSys*, 2025.

Nouha Dziri, Ximing Lu, Melanie Sclar, Xiang Lorraine Li, Liwei Jiang, Bill Yuchen Lin, Peter West, Chandra Bhagavatula, Ronan Le Bras, Jena D. Hwang, et al. Faith and fate: Limits of transformers on compositionality. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

Lizhe Fang, Yifei Wang, Zhaoyang Liu, Chenyang Zhang, Stefanie Jegelka, Jianfeng Gao, Bolin Ding, and Yisen Wang. What is wrong with perplexity for long-context language modeling? In *International Conference on Learning Representations (ICLR)*, 2025.

Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. PAL: Program-aided language models. In *Proceedings of ICML*, 2023a.

Tianyu Gao, Howard Yen, Jiatong Yu, and Danqi Chen. Enabling large language models to generate text with citations. In *Proceedings of EMNLP*, 2023b. Introduces the ALCE benchmark.

Alex J. Goodell, Simon N. Chu, Dara Rouholiman, and Larry F. Chu. Large language model agents can use tools to perform clinical calculations. *npj Digital Medicine*, 8(1):163, 2025. doi: 10.1038/s41746-025-01475-8.

Boyu Gou, Ruohan Wang, Boyuan Zheng, Yanan Xie, Cheng Chang, Yiheng Shu, Huan Sun, and Yu Su. Navigating the digital world as humans do: Universal visual grounding for GUI agents. In *International Conference on Learning Representations (ICLR)*, 2025. ICLR 2025 Oral. Introduces the UGround model.

Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. CRITIC: Large language models can self-correct with tool-interactive critiquing. In *International Conference on Learning Representations (ICLR)*, 2024a.

Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Minlie Huang, Nan Duan, and Weizhu Chen. ToRA: A tool-integrated reasoning agent for mathematical problem solving. In *International Conference on Learning Representations (ICLR)*, 2024b.

Dadi Guo, Jiayu Liu, Zhiyuan Fan, Zhitao He, Haoran Li, Yuxin Li, Yumeng Wang, and Yi R. Fung. Mathematical proof as a litmus test: Revealing failure modes of advanced large reasoning models. *arXiv preprint*, 2025. Introduces the RFMDataset (Reveal Failure Modes).

Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. RULER: What's the real context size of your long-context language models? In *Conference on Language Modeling (COLM)*, 2024.

Dong Huang, Qingwen Bu, Jie M. Zhang, Michael Luck, and Heming Cui. AgentCoder: Multi-agent-based code generation with iterative testing and optimisation. *arXiv preprint*, 2023.

Nikhil Kandpal, Haikang Deng, Adam Roberts, Eric Wallace, and Colin Raffel. Large language models struggle to learn long-tail knowledge. In *Proceedings of ICML*, 2023.

Batuhan K. Karaman, Ishmam Zabir, Alon Benhaim, Vishrav Chaudhary, Mert R. Sabuncu, and Xia Song. POROver: Improving safety and reducing overrefusal in large language models with overgeneration and preference optimization. *arXiv preprint*, 2024.

Marzena Karpinska, Katherine Thai, Kyle Lo, Tanya Goyal, and Mohit Iyyer. One thousand and one pairs: A "novel" challenge for long-context language models. In *Proceedings of EMNLP*, 2024.

Yuri Kuratov, Aydar Bulatov, Pavel Anokhin, Ivan Rodkin, Dmitry Sorokin, Artyom Sorokin, and Mikhail Burtsev. BABILong: Testing the limits of LLMs with long context reasoning-in-a-haystack. In *NeurIPS Datasets and Benchmarks Track*, 2024.

Thomas Kwa et al. Measuring AI ability to complete long software tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. METR technical report.

Yifan Le. Schema key wording as an instruction channel in structured generation under constrained decoding. *arXiv preprint*, 2026.

Yann LeCun. Auto-regressive LLMs are doomed. Tweet, March 26, 2023. `https://x.com/ylecun/status/1640122342570336267`, 2023.

Linzhang Li, Yixin Dong, Guanjie Wang, Ziyi Xu, Alexander Jiang, and Tianqi Chen. XGrammar-2: Efficient dynamic structured generation engine for agentic LLMs. *arXiv preprint*, 2026.

Xuyang Li and Anand D. Sarwate. Unraveling the localized latents: Learning stratified manifold structures in LLM embedding space with sparse mixture-of-experts. *arXiv preprint*, 2025.

Yujia Li et al. Competition-level code generation with AlphaCode. *Science*, 378(6624):1092–1097, 2022. doi: 10.1126/science.abq1158.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *International Conference on Learning Representations (ICLR)*, 2024.

Fangyu Liu, Julian Martin Eisenschlos, Francesco Piccinno, Syrine Krichene, Chenxi Pang, Kenton Lee, Mandar Joshi, Wenhu Chen, Nigel Collier, and Yasemin Altun. DePlot: One-shot visual language reasoning by plot-to-table translation. In *Findings of ACL*, 2023.

Yadong Lu, Jianwei Yang, Yelong Shen, and Ahmed Awadallah. OmniParser for pure vision based GUI agent. *arXiv preprint*, 2024.

Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Daniel Khashabi, and Hannaneh Hajishirzi. When not to trust language models: Investigating effectiveness of parametric and non-parametric memories. In *Proceedings of ACL*, 2023. Introduces the PopQA benchmark.

Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008. Section on Heaps' law: `https://nlp.stanford.edu/IR-book/html/htmledition/heaps-law-estimating-the-number-of-terms-1.html`.

Ayana Niwa and Hayate Iso. AmbigNLG: Addressing task ambiguity in instruction for NLG. In *Proceedings of EMNLP*, 2024.

OpenAI. Introducing structured outputs in the API. OpenAI Engineering Blog. `https://openai.com/index/introducing-structured-outputs-in-the-api/`, 2024. Published August 6, 2024.

OpenAI. GPT-5 system card. `https://cdn.openai.com/gpt-5-system-card.pdf`, 2025. Published August 7, 2025.

Jianhui Pang, Fanghua Ye, Derek F. Wong, Xun He, Wenxiang Chen, and Longyue Wang. Anchor-based large language models. In *Findings of ACL*, 2024.

Kiho Park, Yo Joong Choe, and Victor Veitch. The linear representation hypothesis and the geometry of large language models. In *Proceedings of ICML*, 2024.

Khush Patel, Siva Surendira, Jithin George, and Shreyas Kapale. The six sigma agent: Achieving enterprise-grade reliability in LLM systems through consensus-driven decomposed execution. *arXiv preprint*, 2026.

Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A. Smith, and Mike Lewis. Measuring and narrowing the compositionality gap in language models. In *Findings of EMNLP*, 2023.

Z. Z. Ren et al. DeepSeek-Prover-V2: Advancing formal mathematical reasoning via reinforcement learning for subgoal decomposition. *arXiv preprint*, 2025.

Rylan Schaeffer et al. How do large language monkeys get their power (laws)? In *International Conference on Machine Learning (ICML)*, 2025.

Yu Shang, Yu Li, Keyu Zhao, Likai Ma, Jiahe Liu, Fengli Xu, and Yong Li. AgentSquare: Automatic LLM agent search in modular design space. *arXiv preprint*, 2024.

Yuling Shi, Songsong Wang, Chengcheng Wan, Min Wang, and Xiaodong Gu. From code to correctness: Closing the last mile of code generation with hierarchical debugging. *arXiv preprint*, 2024. Introduces the MGDebugger method.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

Yuhong Sun, Zhangyue Yin, Xuanjing Huang, Xipeng Qiu, and Hui Zhao. Error classification of large language models on math word problems: A dynamically adaptive framework. In *arXiv preprint*, 2025. Introduces the MWPES-300K dataset (304,865 error samples from 15 LLMs across four MWP datasets).

Tarun Suresh et al. DINGO: Constrained inference for diffusion LLMs. *arXiv preprint*, 2025.

Manan Suri, Puneet Mathur, Nedim Lipka, Franck Dernoncourt, Ryan A. Rossi, and Dinesh Manocha. Structured uncertainty guided clarification for LLM agents. *arXiv preprint*, 2025. Introduces the SAGE-Agent method.

Runchu Tian, Yining Ye, Yujia Qin, Xin Cong, Yankai Lin, Zhiyuan Liu, and Maosong Sun. DebugBench: Evaluating debugging capability of large language models. In *Findings of ACL*, 2024.

Akihiko Wada et al. Retrieval-augmented generation elevates local LLM quality in radiology contrast media consultation. *npj Digital Medicine*, 8(1):395, 2025. doi: 10.1038/s41746-025-01802-z.

Kai Wan et al. A Fano-style accuracy upper bound for LLM single-pass reasoning in multi-hop QA. In *International Conference on Learning Representations (ICLR)*, 2026.

Benlu Wang et al. From scores to steps: Diagnosing and improving LLM performance in evidence-based medical calculations. In *Proceedings of EMNLP*, 2025a. Introduces the MedRaC benchmark.

Darren Yow-Bang Wang, Zhengyuan Shen, Soumya Smruti Mishra, Zhichao Xu, Yifei Teng, and Haibo Ding. SLOT: Structuring the output of large language models. *arXiv preprint*, 2025b. SLOT = Structured LLM Output Transformer.

Minzheng Wang et al. Leave no document behind: Benchmarking long-context LLMs with extended multi-doc QA. In *Proceedings of EMNLP*, 2024a.

Peiyi Wang, Lei Li, Zhihong Shao, R. X. Xu, Damai Dai, Yifei Li, Deli Chen, Y. Wu, and Zhifang Sui. Math-Shepherd: Verify and reinforce LLMs step-by-step without human annotations. In *Proceedings of ACL*, 2024b.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

Yubo Wang et al. MMLU-Pro: A more robust and challenging multi-task language understanding benchmark. In *NeurIPS Datasets and Benchmarks Track (Spotlight)*, 2024c.

Hao Wen et al. Fixing function-level code generation errors for foundation large language models. *arXiv preprint*, 2024. Introduces the LlmFix method. Reports AssertionError 63.64% + NameError 22.71% = 86.35% of HumanEval failures across 14 LLMs.

Michael C. Wood and Adam A. Forbes. 100% elimination of hallucinations on RAGTruth for GPT-4 and GPT-3.5 Turbo. *arXiv preprint*, 2024. Introduces the Acurai method.

Tianbao Xie et al. Scaling computer-use grounding via user interface decomposition and synthesis. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. NeurIPS 2025 Spotlight. Introduces the Jedi grounding dataset and OSWorld-G benchmark.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint*, 2024.

Cyril Zakka, Rohan Shad, Akash Chaurasia, Alex R. Dalal, Jennifer L. Kim, Michael Moor, Robyn Fong, Curran Phillips, Kevin Alexander, Euan Ashley, Jack Boyd, Kathleen Boyd, Karen Hirsch, Curtis Langlotz, Rita Lee, Joanna Melia, Joanna Nelson, Karim Sallam, Stacey Tullis, Melissa Ann Vogelsong, John Patrick Cunningham, and William Hiesinger. Almanac — retrieval-augmented language models for clinical medicine. *NEJM AI*, 1(2):AIoa2300068, 2024. doi: 10.1056/AIoa2300068.

Kexun Zhang et al. Don't fine-tune, decode: Syntax error-free tool use via constrained decoding. *arXiv preprint*, 2023. Introduces the ToolDec method.

Meiru Zhang, Zaiqiao Meng, and Nigel Collier. Failure modes in multi-hop QA: The weakest link effect and the recognition bottleneck. *arXiv preprint*, 2026.

Yang Zhou, Hongyi Liu, Zhuoyan Chen, Yuandong Tian, and Beidi Chen. GSM-Infinite: How do your LLMs behave over infinitely increasing context length and reasoning complexity? *arXiv preprint*, 2025.

A Formal Proofs, Derivations, and Sensitivity Analysis

A.1 Proof of Proposition 1: No Universal Finite Intervention Dictionary

Fix a residual-error tolerance ε. Say that an intervention *covers* a failure mode if it reduces the residual error of that mode below ε. Coverage is mode-level: an intervention that covers a mode covers every failure event in that mode. The two conditions "D is intervention-unbounded" and "$|C_D^\varepsilon| = \infty$" are equivalent under mode-level coverage; an unbounded witnessing sequence is constructed by taking one representative per mode, and an infinite catalogue forces the existence of such a sequence.

Let D be a domain. Call D *intervention-unbounded* if it contains an infinite sequence of failures $f_1, f_2, f_3, \ldots$ such that each new f_j is not covered by any finite intervention dictionary that covers all earlier failures $\{f_1, \ldots, f_{j-1}\}$.

Step 1: assume the opposite. Suppose, for contradiction, that C_D^{ε} is finite. Then there are only finitely many intervention-distinguishable failure modes in D. Write them as $C_D^{\varepsilon} = \{c_1, \ldots, c_M\}$ for some finite M.

Step 2: what finiteness means. If there are only M intervention-distinguishable modes, then after all M modes have appeared in the sequence, every later failure must belong to one of the already-seen modes.

Step 3: same mode means same intervention class. Modes are defined at intervention resolution ε. If a later failure belongs to the same mode as an earlier failure, then the intervention dictionary that covers the earlier representative of that mode also covers the later failure below residual tolerance ε.

Step 4: contradiction. Intervention-unboundedness says exactly the opposite: each new f_j is not covered by any finite dictionary that covers $\{f_1, \ldots, f_{j-1}\}$. Therefore f_j cannot belong to any earlier intervention mode, so each f_j introduces a new intervention-distinguishable mode. The sequence $f_1, f_2, f_3, \ldots$ induces infinitely many such modes, contradicting the assumption that C_D^{ε} is finite. Hence $|C_D^{\varepsilon}| = \infty$.

Step 5: no finite dictionary covers the domain. Suppose, again for contradiction, that some finite intervention dictionary $\mathcal{I}$ covers all of D. Then $\mathcal{I}$ covers every finite prefix $\{f_1, \ldots, f_{j-1}\}$ for every j. By intervention-unboundedness, any dictionary covering that prefix fails to cover f_j. Therefore $\mathcal{I}$ does not cover f_j, contradicting the assumption that $\mathcal{I}$ covers all of D. $\square$

A.2 Derivation of Proposition 2: Patch-Local Intervention Budget

Conditions used. Proposition 2 depends on four assumptions:

1. **Coverage model.** The cumulative hard-error mass covered by the top m modes is approximated by
$$F(m; |C|) \;=\; \min\!\left(1,\; \frac{\ln m}{\ln |C|}\right)$$
on the declared domain $m \geq 2$, $|C| \geq 2$.
2. **Non-trivial, attainable target.** The residual target satisfies $\varepsilon \in (0, e_{\text{hard}})$. If $\varepsilon \geq e_{\text{hard}}$ no intervention is needed; if $\varepsilon \leq 0$ the target is unattainable unless all residual hard-token error is eliminated.
3. **Patch-local catalogue.** The result applies only after a deployment patch D has been fixed and its reachable catalogue is modelled as finite or effectively capped.
4. **Sequence-length claim.** The doubly-logarithmic rate further requires Assumption 2 and $k(n) = \Theta(\log n)$. Without these, Proposition 2 still gives a catalogue-size budget but not the same sequence-length scaling.

Step 1: define the target. Let e_{hard} be the baseline hard-token error rate and $\varepsilon \in (0, e_{\text{hard}})$ the target after intervention.

Step 2: define the effective catalogue. For per-sequence scaling, set $C_{\text{eff}} = C_{\text{active},D}(n)$: how many failure modes can a single sequence of length n activate? For full deployment-library budgeting, set $C_{\text{eff}} = C_D$: how large must the library be to cover the recurring failure modes reachable inside D?

Step 3: cumulative coverage. Let the ranked local failure modes have hard-error masses $p_1 \geq p_2 \geq \cdots \geq p_{|C_{\text{eff}}|}$ with $\sum_i p_i = 1$. A library covering the top m modes removes cumulative hard-error mass $F(m; |C_{\text{eff}}|) = \sum_{i=1}^{m} p_i$, approximated by the log-coverage form of §3.3.

Step 4: residual after intervention. The uncovered mass fraction is $1 - F(m; |C_{\text{eff}}|)$, so the residual per-hard-token error rate is $e_{\text{res}}(m) = (1 - F(m; |C_{\text{eff}}|))\, e_{\text{hard}}$.

Step 5: impose the target. Requiring $e_{\text{res}}(m) \le \varepsilon$ and dividing by $e_{\text{hard}} > 0$ gives

$$1 - F(m; |C_{\text{eff}}|) \;\le\; \frac{\varepsilon}{e_{\text{hard}}}, \qquad \text{i.e.,} \qquad F(m; |C_{\text{eff}}|) \;\ge\; 1 - \frac{\varepsilon}{e_{\text{hard}}}.$$

Step 6: substitute the log-coverage form. Since $\varepsilon < e_{\text{hard}}$, the required coverage $1 - \varepsilon/e_{\text{hard}} \in (0, 1)$, so the cap in F is inactive before saturation. Using $F = \ln m / \ln |C_{\text{eff}}|$:

$$\frac{\ln m}{\ln |C_{\text{eff}}|} \;\ge\; 1 - \frac{\varepsilon}{e_{\text{hard}}}.$$

Step 7: solve for m. Multiplying by $\ln |C_{\text{eff}}| > 0$ and exponentiating,

$$m \;\ge\; \exp\left[\left(1 - \frac{\varepsilon}{e_{\text{hard}}}\right) \ln |C_{\text{eff}}|\right] \;=\; |C_{\text{eff}}|^{1-\varepsilon/e_{\text{hard}}}.$$

Taking the ceiling (since m is integer-valued) yields $m \ge \lceil |C_{\text{eff}}|^{1-\varepsilon/e_{\text{hard}}} \rceil$, the bound stated in Eq. (4).

Step 8: sequence-length rate. For per-sequence scaling, set $C_{\text{eff}} = C_{\text{active},D}(n)$. Assumption 2 bounds $|C_{\text{active},D}(n)| \le \min(A'_D + \sigma'_D \ln h(n),\, |C_D|)$. In the pre-cap regime, i.e., while $A'_D + \sigma'_D \ln h(n) < |C_D|$, the ceiling has not been reached and $|C_{\text{active},D}(n)| = O(\ln h(n))$. With $h(n) = \beta k(n)$ and $k(n) = \Theta(\log n)$, we get $h(n) = \Theta(\log n)$ and $\ln h(n) = \Theta(\log \log n)$, hence

$$|C_{\text{active},D}(n)| \;=\; O(\log \log n),$$

and substituting into the budget,

$$m \;=\; O\big((\log \log n)^{1-\varepsilon/e_{\text{hard}}}\big).$$

This is the doubly-logarithmic pre-cap special case.

Step 9: cap regime. Once active-mode discovery saturates the patch catalogue, $|C_{\text{active},D}(n)| = |C_D|$, so $m \ge \lceil |C_D|^{1-\varepsilon/e_{\text{hard}}} \rceil$, which no longer depends on n. The intervention budget is then domain-constant. □

A.3 Sequence-Level Reliability Derivation

Proposition 2 gives a per-hard-token residual target. Production systems often care about a stricter target: the probability that the entire sequence is correct.

Starting from the composed reliability of Eq. (1), and writing the post-intervention hard-token rate as e_{res},

$$P(\text{correct}) \;=\; (1 - e_{\text{res}})^{\beta k}\, (1 - e_{\text{easy}})^{(1-\beta)k}\, (1 - e_{\text{non}})^{n-k}.$$

Define the non-hard-token survival factor

$$S_{\text{base}} \;=\; (1 - e_{\text{easy}})^{(1-\beta)k}\, (1 - e_{\text{non}})^{n-k}, \tag{5}$$

so that $P(\text{correct}) = (1 - e_{\text{res}})^{\beta k} S_{\text{base}}$. The sequence-level target $P(\text{correct}) \ge 1 - \varepsilon_{\text{seq}}$ becomes

$$(1 - e_{\text{res}})^{\beta k} S_{\text{base}} \;\ge\; 1 - \varepsilon_{\text{seq}}, \qquad \text{i.e.,} \qquad (1 - e_{\text{res}})^{\beta k} \;\ge\; \frac{1 - \varepsilon_{\text{seq}}}{S_{\text{base}}}.$$

Three regimes follow.

Regime (i): non-hard-token errors already violate the target. If $S_{\text{base}} < 1 - \varepsilon_{\text{seq}}$, then even setting $e_{\text{res}} = 0$ cannot meet the target, because the maximum possible survival after eliminating all hard-token failures is only S_{base}. Hard-token interventions alone cannot meet the sequence-level SLA. This is the route by which the exponential-in-n concern re-enters and should be diagnosed before any catalogue-budgeting exercise.

Regime (ii): hard-token residual error determines feasibility. If $S_{\text{base}} \geq 1 - \varepsilon_{\text{seq}}$, the target may be feasible. Taking the $(1/(\beta k))$-th power of the rearranged inequality and isolating e_{res},

$$e_{\text{res}} \;\leq\; 1 - \left(\frac{1-\varepsilon_{\text{seq}}}{S_{\text{base}}}\right)^{1/(\beta k)} \;=:\; \tau_{\text{seq}}.$$

Applying Proposition 2 with ε replaced by τ_{seq},

$$m \;\geq\; \left\lceil |C_{\text{eff}}|^{1-\tau_{\text{seq}}/e_{\text{hard}}} \right\rceil.$$

As the sequence-level target becomes stricter (ε_{seq} shrinks), $\tau_{\text{seq}} \to 0$ and the exponent $1 - \tau_{\text{seq}}/e_{\text{hard}} \to 1$, so $m \to |C_{\text{eff}}|$. Strict one-shot sequence-level reliability pushes the system toward full-catalogue coverage.

Regime (iii): baseline hard-token error is already acceptable. If $\tau_{\text{seq}} \geq e_{\text{hard}}$, the baseline hard-token rate already satisfies the sequence target (no intervention is required because $e_{\text{res}}(0) = e_{\text{hard}}$ from Eq. (3)).

Conclusion. Per-hard-token reliability is easier than sequence-level reliability. A library that gives a large reduction in residual hard-token error may still be insufficient for one-shot sequence-level guarantees when many hard decisions occur in a single output. This is why the main paper treats the "≈ 50 patterns" rule of thumb as a per-hard-token planning prior, not a one-shot sequence-level SLA.

A.4 Why these are propositions rather than unconditional theorems

Proposition 1 is a definitional impossibility result: once intervention-unboundedness is assumed, an infinite intervention-resolution catalogue follows. Its role is not to prove that every unbounded domain necessarily has infinite failure modes, but to show that open-ended domains cannot be assumed to admit finite dictionaries.

Proposition 2 is conditional engineering math. It does not prove that LLM failures universally obey logarithmic mode discovery. It proves that *if* a bounded patch has a finite or effectively capped reachable catalogue, *and if* cumulative intervention coverage follows the stated head-heavy form, *then* the required per-hard-token intervention budget grows slowly and becomes constant after catalogue saturation.

The empirical burden therefore lies not in the algebra but in measuring, for each deployment patch, the local discovery curve $C_{\text{seen},D}(T)$, the per-sequence activation $C_{\text{active},D}(n)$, the cumulative coverage $F(m; |C_D|)$, the hard-token fraction β_D, and the baseline hard-token rate e_{hard}. This is why the paper frames the results as a reliability-engineering scaffold rather than as universal theorems about LLM behaviour.

A.5 Heaps Power-Law Variant and Cluster-Count Sensitivity

A reader who prefers to derive cluster-count growth from standard Heaps' law rather than from Postulate 1 obtains a qualitatively similar result. Let $|C|(k_{\text{hard}}) \approx K \cdot k_{\text{hard}}^b$ with $b \in (0,1)$. Canonical fits give $b \approx 0.5$ for natural-language vocabularies; failure-mode taxonomies plateau much more sharply (ErrorAtlas stabilises at $|C| = 17$ across 10^4+ failures), implying a smaller effective $b \approx 0.2$ in our setting. Composing with $k = \Theta(\log n)$, we get $|C| = O((\log n)^b)$, and Proposition 2 becomes

$$m = O\Big((\log n)^{b\cdot(1-\varepsilon/e_{\text{hard}})}\Big).$$

This is still polylogarithmic in n for any $b \in (0,1)$ and any $\varepsilon < e_{\text{hard}}$. The paper's qualitative claim survives either choice of cluster-count law.

Symbolic-form sensitivity across candidate laws. The polylog conclusion depends on which cluster-count law one accepts; available evidence is consistent with multiple candidates because no subsample-discovery curve has been published for any LLM failure-mode taxonomy at this writing. We therefore report symbolic rates rather than fitted constants. With $h(n) = \beta k(n)$:

- **Logarithmic:** $|C_{\text{active},D}(n)| = O(\log h(n))$. If $k(n) = \Theta(\log n)$, then $m = O\big((\log\log n)^{1-\varepsilon/e_{\text{hard}}}\big)$.
- **Heaps:** $|C_{\text{active},D}(n)| = O\big(h(n)^b\big)$ with $b \in (0,1)$. If $k(n) = \Theta(\log n)$, then $m = O\big((\log n)^{b\,(1-\varepsilon/e_{\text{hard}})}\big)$.
- **Saturating:** $|C_{\text{active},D}(n)| \le |C_D|$. Then $m = O\big(|C_D|^{1-\varepsilon/e_{\text{hard}}}\big)$, constant in n once the patch ceiling is reached.

The qualitative conclusion is robust **under the** $k(n) = \Theta(\log n)$ **regime**: m grows more slowly than any positive power of n under every candidate, and the directional claim ("a small library covers the head of the failure distribution in the per-hard-token regime") survives. Only the exponent shifts: doubly-logarithmic under logarithmic discovery, $(\log n)^b$ with small b under Heaps, constant in the cap regime. The doubly-logarithmic rate is the optimistic special case. If $k(n)$ grows as a positive power of n, the Heaps variant inherits that power and the polylog-in-n language fails along that axis; the framework's intervention prescription still applies, but its asymptotic-rate framing does not.

Numerical constants require a measured discovery curve $C_{\text{seen},D}(T)$ or $C_{\text{active},D}(n)$. Existing taxonomies provide endpoint category counts at a single corpus scale (ErrorAtlas at $|C| = 17$ for $\approx 10^4$ failures), not discovery curves. The subsample-discovery measurement remains the explicit empirical test that would either tighten the postulate or fall back to the Heaps variant. Until that measurement exists, the framework's headline rate should be read as *polylogarithmic in the pre-cap regime, domain-constant in the cap regime*, with the specific exponent flagged as a falsifiability test rather than a fitted prediction.

B Full Failure-Mode Taxonomy and the Capability-Elimination Harvest

The intervention literature provides at least one targeted countermeasure for each named cluster of §4. Two organising granularities exist: at the capability level (six axes) the cluster-orthogonality property is most clearly visible; at the error-class level (twelve named clusters) the evidence aligns with the taxonomies of §4 but is finer-grained than the underlying capability mechanisms.

Six capability axes and three structural patterns. A dedicated harvest yields **28 quantitatively-anchored citations** across **six independent capability axes**: Arithmetic (Python/symbolic execution), Code Execution (REPL/sandbox feedback), Format/Structure (constrained decoding, FSMs, grammar engines), Perception/Grounding (visual grounding for GUI, charts, tables), Knowledge/RAG (dense retrieval and citation grounding), Verification (proof checkers, learned verifiers, classifier rerouting, process supervision). Each axis is independently confirmed by between three and nine citations. We stratify the 28 by kind of evidence into three patterns:

Pattern A: hard guarantees (by-construction). Seven citations achieve residual error rate equal to zero *by construction*, restricted strictly to structural/verifiable classes: constrained decoders set $P(\text{invalid token}) = 0$ at every step [Suresh et al., 2025, Zhang et al., 2023, Dong et al., 2025, Li et al., 2026, OpenAI, 2024]; static syntax checks reject programs with a `SyntaxError` before execution [Wen et al., 2024]; proof kernels reject any output failing type-checking [Ren et al., 2025]. The class of grammar-violating outputs is mathematically empty under these mechanisms.

Pattern B: strong empirical reductions with class-shift signature. Roughly fourteen citations report empirical reductions of 80–100% in a named error class, with the post-intervention failure log dominated by structurally different residual classes. Program-of-Thoughts on GSM8K [Chen et al., 2023]: calculation errors drop from 30% of failures to 0%, residuals are 62% reasoning + 36% misunderstanding. OpenMedCalc [Goodell et al., 2025]: "only interpretation errors were identified." Acurai [Wood and Forbes, 2024]: 100% hallucination elimination on RAGTruth (95% CI 91–100%), strong empirical rather than by-construction.

Pattern C: moderate reductions (60–80%) with residuals shifting outside the target class. Legal RAG [Dantart, 2026]: fabricated citations $> 30\% \to < 0.2\%$. GPT-5 SimpleQA with web access [OpenAI, 2025]: $47\% \to 9.6\%$ (inter-condition, not within-condition). CRITIC [Gou et al., 2024a]: toxic generation -79.2%.

The 28 citations, organised by axis and pattern.

Axis	Citation & targeted class	Pre $\to$ Post	Pattern
Verification	Ren et al. [2025]: Invalid Lean proof	any $\to$ 0% by constr.	A
Format	OpenAI [2024]: JSON schema violation	$>$60% $\to$ 0% by constr.	A
Format	Zhang et al. [2023]: Tool-call syntax	21–100% $\to$ 0%	A
Format	Suresh et al. [2025]: JSON parse failure	13–82% $\to$ 0%	A
Format	Dong et al. [2025]: Multi-format errors	20–38% $\to$ 0%	A
Format	Li et al. [2026]: Malformed tool calls	33–78% $\to$ 0%	A
Arithmetic	Chen et al. [2023]: Calculation on GSM8K	30% $\to$ 0% of failures	B
Arithmetic	Goodell et al. [2025]: Clinical arithmetic	"only interpretation errors"	B
Knowledge/RAG	Wada et al. [2025]: RAG hallucinations	8% $\to$ 0% ($p = 0.012$)	B
Knowledge/RAG	Wood and Forbes [2024]: Context-conflict hallucinations	100% $\to$ 0% on subset	B
Knowledge/RAG	Dantart [2026]: Fabricated legal citations	$>$30% $\to$ $<$0.2%	C
Code Exec	Wen et al. [2024]: `SyntaxError` (HumanEval)	5.76% $\to$ 0.01%	A
Code Exec	Li et al. [2022]: False-positive submissions	62% $\to$ 4%	B
Code Exec	Shi et al. [2024]:5 code-bug classes	100% repair (5/6 classes)	B
Knowledge/RAG	Gao et al. [2023b]: Citation grounding (ELI5)	$\approx$50% w/o complete support	C
Knowledge/RAG	OpenAI [2025]: Factual errors	47% $\to$ 9.6% w/web	C
Knowledge/RAG	Zakka et al. [2024]: Clinical citation errors	44% $\to$ 9%	C
Arithmetic	Wang et al. [2025a]: Arithmetic in medical reasoning	426 $\to$ 74 errors	B
Code Exec	Wen et al. [2024]: `NameError` repair	22.7% $\to$ 2.3%	C
Perception	Gou et al. [2025]: GUI grounding errors	16.2% $\to$ 73.3% acc.	B
Perception	Xie et al. [2025]: GUI task failures	5% $\to$ 27% SR (5.4$\times$)	B
Perception	Cheng et al. [2024]: Element mislocation	5.2% $\to$ 53.4% (10.3$\times$)	B
Perception	Liu et al. [2023]: Chart-reading errors	38.2% $\to$ 67.6% (+29.4 pp)	B
Verification	Gou et al. [2024a]: Toxic generation	$-$79.2%	C
Verification	Wang et al. [2024b]: Reasoning step errors	28.6% $\to$ 43.5% w/rerank	B
Knowledge/RAG	Asai et al. [2024]: Unsupported facts	55.5% $\to$ 22% error	B
Perception	Lu et al. [2024]: Icon/element errors	70.5% $\to$ 93.8% (79% red.)	B
Knowledge/RAG	Mallen et al. [2023]: Long-tail entity errors	$\approx$80% $\to$ $\approx$50% failure	B

Capabilities are coarser than error classes. Several entries in the harvest reveal "two-for-one" reductions where one capability addresses multiple named clusters: a single Python interpreter removes the execution-error component of arithmetic, unit conversion, simple counting, list manipulation, and date arithmetic; constrained decoding eliminates by construction both format violations and the structural component of "missing required element"; code execution feedback strongly reduces `SyntaxError`, `NameError`, and most `TypeError` together; RAG strongly reduces factual hallucinations, fabricated citations, and outdated information jointly. The practical capability library required is therefore smaller than the count of named error categories.

The twelve named clusters.

Cluster	Axis	Intervention	Before → After	Patt.
A. Arithmetic	Arithmetic	PAL Python [Gao et al., 2023a]	20.1% → 61.5%	B
B. Unit conversion	Arithmetic	Folded into A (Python)	n/a	n/a
C. Counting	Arithmetic	Explicit counter (gap)	n/a	gap
D. Format/schema	Format	DINGO/SLOT [Suresh et al., 2025, Wang et al., 2025b]	up to +68 pp; 99.5% acc.	A
E. Code logic/type	Code Exec	Reflexion/AgentCoder [Shinn et al., 2023, Huang et al., 2023]	80% → 96.3% pass@1	A/B
F. Multi-hop drift	Knowledge	Entity-grounded rewriter	≈5–10 pp	C
G. Reasoning step	Verification	Math-Shepherd [Wang et al., 2024b]	28.6% → 43.5%	B
H. Spec misinterpret.	Verification	Clarification [Niwa and Iso, 2024]	ROUGE-L +15	C
I.a Struct. missing	Format	Subsumed by D (constr. decode)	n/a	A
I.b Semantic missing	Verification	Subsumed by G/H	n/a	C
J. Hallucination	Knowledge	RAG [Wood and Forbes, 2024]	non-zero → 0%	B/C
K. Refusal	Verification	POROver [Karaman et al., 2024]	57.6% → 82.1%	B
L. Tool/API	Format+Verif.	SAGE-Agent [Suri et al., 2025]	36.5% → 65.2%	B

Eight of twelve categories have a strong citation with double-digit percentage-point improvement; three (B, C, I) lack a clean single-cluster ablation; one (F) has consistent medium-strength evidence. Category I (missing required elements) splits mechanistically into I.a (structural, absorbed by D's constrained decoders) and I.b (semantic, absorbed by G/H). Categories B (units) and C (counting) remain technical gaps: B is naturally folded into A; C is the smallest residual gap.

Additivity and its limits. Patel et al. [2026] demonstrate that stacking interventions targeting orthogonal failure modes can produce large compound reliability gains: their parallel-consensus framework yields a $14{,}700\times$ improvement over single-pass baseline, evidence for compound gains from decomposition plus consensus aggregation under their specific parallel-voting regime rather than a direct demonstration that heterogeneous cluster-targeted interventions compose additively without voting. Shang et al. [2024] similarly show supra-additive gains when modules target distinct failure modes. Le [2026] reports that schema-level and prompt-level instructions interact *non-additively* when sharing a prompt channel: interventions on orthogonal processing layers (decoding constraint vs. retrieval vs. training-signal vs. inference-time tool call) compose near-additively, while those sharing a channel may interfere.

The irreducible-semantic residual. Of the 17 ErrorAtlas categories, 13 are addressed under Patterns A, B, or C by one of the six capability axes; four are not: residuals of inappropriate refusal beyond preference optimisation, specification misinterpretation not closed by clarification, problem-decomposition reasoning bottlenecks, and a "user wanted something different" semantic remainder. These are classes where the failure is in choosing what to do, not executing it. Proposition 2's prediction of $O(\log)$ rather than $O(0)$ residual reliability reflects exactly this irreducible core. The framework does not claim 100% coverage of all failures; it claims polylog-bounded *capability-eliminable* failures, with the named semantic residual as the floor.

C Counter-Evidence Re-Audits

Five prominent papers are routinely cited as evidence that LLM reliability decays steeply with length [Dziri et al., 2023, Kuratov et al., 2024, Kwa et al., 2025, Wan et al., 2026, Karpinska et al., 2024]. A careful re-reading shows that *every one* of these papers decays over a variable distinct from raw token length n. Identifying the decay axis is not the same as dissolving the practical concern: where compositional graph size, fact count, or evidence scope grow with problem length, the framework predicts steep failure curves. The contribution is to identify which interventions help (capability provisioning along the actual decay axis) and which do not.

Dziri et al. [2023], Faith and Fate. GPT-4 multi-digit multiplication accuracy drops from 59% (3-digit) to 4% (4-digit) zero-shot, with the authors theorising "probability of incorrect predictions converges exponentially to ≈ 1 for abstract compositional tasks." The decay variable is *compositional graph size* N, not raw token length: the 3×3 graph has on the order of d^2 partial products plus carries, and multi-digit multiplication is engineered so that $k_{\text{hard}} \approx N$, with every node in the computation graph a hard decision. For natural-language tasks where $k_{\text{hard}} \ll n$, Dziri's regime is the *boundary case* in which our framework reduces to their result. A clean $(1-\varepsilon)^N$ exponential cannot simultaneously reproduce 59% at 3-digit and 4% at 4-digit for any single per-node ε: the observed drop is locally steeper than per-node iid exponential, consistent with a finite catalogue of failure modes exhausting as N grows. *Where the framework concedes*: in adversarial compositional tasks engineered so $k_{\text{hard}} \approx N$, the framework reduces to Dziri's regime and does not relieve it; the relocation is informational, not magical.

Kuratov et al. [2024]. The abstract reports "performance declines sharply with increased reasoning complexity" and models "effectively utilise only 10–20% of the context." Read in detail, BABILong varies two axes: context length n (0K to 10M tokens) and number of supporting facts k (QA1 $= 1$ fact to QA3 $= 3$ facts). The sharp decline is in k, not n. For QA1 (single-fact), most models "perform well up to 4,000 tokens": a plateau, not exponential decay. The famous RAG-flat-across-length result (60% on single-fact QA *independent of context length*) is the cleanest possible demonstration that when relevant evidence is in window, length does not matter. Recurrent Memory Transformers maintaining performance to 50M tokens further confirms effective k is determined by architecture, not raw n. The framework's concession is narrow but real: for multi-hop tasks whose required fact count grows with task complexity, BABILong's sharp k-axis decay is exactly what the framework predicts happens, not a counter-example.

Kwa et al. [2025] (METR). Per-model success fits a logistic in $\log(\text{human-task-duration})$: $S(t) = \sigma(\beta(\log t - \log h))$. Logistic-in-log-length is *mathematically sublinear* in length itself. The 80%-horizon being 4–6$\times$ shorter than the 50%-horizon is a steep within-model cliff but compatible by construction with the polylog result: a cliff at a specific capacity threshold is a manifold-transition signature. The famous exponential (capability doubling every seven months) is an *inter-model* claim about how the horizon h moves across generations, orthogonal to within-model decay shape. An honest qualifier: the within-model logistic-in-log cliff is steep on a practical scale. Calling it "sublinear in length" is technically correct but engineering-useful only for systems designed to operate well below the 50% horizon.

Wan et al. [2026], Fano-style upper bound. This paper theorises super-linear information-demand growth and identifies an "accuracy cliff" at capacity overflow: "when the task's information demand surpasses the model's output capacity, performance does not degrade gracefully but instead collapses sharply." The behavioural prediction is a *threshold*, not smooth $(1-\varepsilon)^n$. A cliff at a specific capacity threshold is exactly the manifold-transition behaviour our framework predicts at the boundary between covered and uncovered modes; Wan et al.'s mechanism (capacity overflow) is one specific cause, complementary to ours. *Where the framework concedes*: Wan documents a mechanism by which $|C|$ effectively exceeds the model's coverage in a single forward pass; Pattern A interventions (constrained decoding, formal verification) are the kind of response the framework expects but does not automatically supply.

Karpinska et al. [2024]. GPT-4o achieves 55.8% pair accuracy across 1,001 minimally-different true/false claim pairs about long fictional books (mean length 127K tokens). The paper reports performance by *evidence scope*, not by context length: 59.8% on sentence-level retrieval, 47.6% on passage-level, 41.6% on global reasoning. No per-context-length curve is reported. The decay axis is the number of evidence pieces that must be integrated, a k-axis quantity rather than an n-axis one. NoCha is therefore not counter-evidence to a sublinear-in-n claim. Evidence-scope decay is exactly the k-axis observation the framework predicts cannot be addressed by scaling raw context length; it requires retrieval or process supervision along the actual decay axis. The relocation is informational, not magical.

Unifying observation. Every counter-paper decays over a variable that is not n: compositional graph size, fact count, log-time horizon, capacity threshold, or evidence scope. The apparent rapid decay is, in each case, in a quantity our framework already concentrates the action in (k_{hard} and $|C|$), not in raw sequence length. This is a relocation, not a dissolution. Where k_{hard} grows with task length (adversarial compositional structure, multi-hop fact chains, long horizons that force more decisions), reliability remains hard; the framework's value is directing intervention toward capability provisioning along the actual decay axis rather than toward context-window or compute-budget expansion that does not help.

D Patch Evidence Detail

The body's claim that domain patches cap the engineering problem rests on a triangulation of indirect evidence. None of the items below *measures* the patch ceiling $|C_D|$ directly; they support the weaker claim that model behaviour is strongly domain-dependent and that operational neighbourhoods occupy bounded regions of the model's behavioural space.

Evidence type	**What it supports**	**What it does not prove**
Low intrinsic-dimensional structure in representations [Park et al., 2024, Li and Sarwate, 2025]	Domains occupy localised structure in model representation space	Does not measure failure-catalogue size $|C_D|$
Cross-domain accuracy spreads on a single base model [Wang et al., 2024c]	Same model behaves differently across domains; performance is patch-dependent	Does not imply finite failure modes
Long-tail / popularity thresholds in knowledge tasks [Mallen et al., 2023, Kandpal et al., 2023]	Coverage depends on domain frequency and training exposure	Does not prove logarithmic mode discovery
Patch-specific intervention performance (across §4, Appendix B)	Local tools, schemas, and capability libraries change residual error structure	Does not imply universal cross-patch transfer of the same intervention library

The purpose of this evidence is motivational. It supports patch-indexing of σ_D, A_D, β_D, $|C_D|$, but the finite (or effectively capped) reachable catalogue remains an empirical modelling assumption to be measured per deployment, not derived from any of the rows above.

E Empirical Calibration Detail

Three published LLM error taxonomies anchor the mode-rate parameter σ in the body's $\sigma \in [0.87, 1.85]$ range. The simple calibration uses $|C| \approx A + \sigma \ln T$ with $A = 0$; this is a deliberately conservative readout that ignores any positive intercept and uses endpoint counts rather than discovery curves.

Source	**Approx. observed failures T**	**Named categories $|C|$**	**Implied σ at $A = 0$**	**Role**
ErrorAtlas [Ashury-Tahan et al., 2026]	$\gtrsim 10^4$	17	≈ 1.85	Conservative cross-domain anchor (used as planning value)
HumanEval categorisation [Wen et al., 2024]	comparable scale	8–12	≈ 0.87–1.30	Code-domain anchor
MWPES-300K [Sun et al., 2025]	$\approx 3 \times 10^5$	15–20	≈ 1.2–1.6	Math-domain anchor (largest corpus)

These are endpoint counts, not discovery curves: they tell us *how many* named categories a taxonomer assigned at a single corpus scale, not how the count grew with T. They do not prove logarithmic mode discovery. Their role in this paper is twofold: they motivate the empirical postulate of §3.2, and they identify the explicit empirical test that would either tighten the postulate or move the analysis to the Heaps variant of Appendix A.5: repeatedly subsample failures from a fixed deployment patch D and plot discovered modes $|C_{\text{seen},D}(T)|$ against T.